PENETRATION TEST

& THREAT INTELLIGENCE ASSESSMENT REPORT

Target Environment: “Bricks Heist” — Internet-Facing WordPress / Bricks Builder Web Server

Unauthenticated RCE Exploitation → Discovery of Pre-Existing Ransomware-Affiliated Compromise

Lab Platform: TryHackMe | Room: TryHack3M: Bricks Heist

Prepared For: Client / Portfolio Demonstration Engagement

Prepared By: Offensive Security Assessment Team

Lab Platform: TryHackMe (thm.io) — Room Name: “TryHack3M: Bricks Heist”

Engagement Type: Authorized Penetration Test + Digital Forensics / Threat Intelligence — TryHackMe Lab / Controlled Environment

Assessment Window: September 3, 2026

Report Date: September 3, 2026

Report Classification: CONFIDENTIAL — For Intended Recipient Only

1. Executive Summary

This report documents a penetration test performed against “Bricks Heist,” a TryHackMe lab room hosting an internet-facing WordPress installation built on the Bricks Builder theme. The objective was to identify externally exploitable vulnerabilities, gain a foothold, and assess post-exploitation impact. Unlike a typical clean lab build, this engagement uncovered something notable during post-exploitation enumeration: clear indicators that the host had already been compromised by a separate, simulated threat actor prior to this assessment, including a disguised persistence mechanism, an obfuscated cryptocurrency wallet artifact, and a planted file in the web root.

Initial access was obtained by exploiting a known, publicly disclosed unauthenticated Remote Code Execution vulnerability in the Bricks Builder WordPress theme (CVE-2024-25600), yielding command execution as the apache service account. Enumeration of the compromised host then revealed a WordPress configuration file disclosing plaintext database credentials and application secrets, an exposed MySQL service, a suspicious systemd service masquerading under an innocuous name, and a hidden artifact file planted in the web root by what appears to be a prior intrusion. A cryptocurrency wallet address recovered from the host — stored under multiple layers of encoding — was cross-referenced against open-source threat intelligence and found to correspond to a wallet publicly linked to a sanctioned individual associated with the LockBit ransomware group.

Overall Risk Rating: CRITICAL. This engagement demonstrates both an exploitable unauthenticated RCE vulnerability and evidence consistent with a real prior compromise of the environment. Both the vulnerability itself and the artifacts of the pre-existing intrusion require immediate attention: the former to prevent re-exploitation, and the latter to support incident response, eradication, and threat-intelligence reporting.

1.1 Engagement Snapshot

#	Finding	Severity	CVSS-Style Risk
1	Unauthenticated Remote Code Execution — Bricks Builder Theme (CVE-2024-25600)	**Critical**	9.8 (Critical)
2	Outdated WordPress Core (6.5) & Bricks Theme (1.9.5)	**High**	7.5 (High)
3	Sensitive Configuration Disclosure — wp-config.php (DB Credentials & Secret Keys)	**Critical**	9.1 (Critical)
4	Exposed MySQL Service Combined with Weak Database Credentials	**High**	8.2 (High)
5	Evidence of Pre-Existing Compromise — Masquerading Persistence Service	**Critical**	9.0 (Critical)
6	Planted Hidden Artifact File in Web Root (Root-Owned)	**High**	7.2 (High)
7	Obfuscated Cryptocurrency Wallet Artifact Recovered from Host	**High**	7.0 (High)
8	Threat-Intelligence Correlation — Wallet Linked to Sanctioned LockBit-Affiliated Actor	**Critical**	N/A (CTI Finding)
9	VPN / Tunneling Utilities Present Under System Network Manager Path	**Medium**	5.5 (Medium)

#	Finding	Severity	CVSS-Style Risk
10	WordPress User Enumeration	Medium	5.3 (Medium)
11	Administrative Interfaces Exposed (phpMyAdmin, wp-admin) via Predictable Paths	Medium	5.8 (Medium)

2. Scope & Objectives

2.1 Scope of Work

Lab Platform: TryHackMe (thm.io) — Room Name: “TryHack3M: Bricks Heist”

Target Host: bricks.thm — 10.48.170.53 (TryHackMe-provisioned lab network)

In-Scope Services: TCP/22 (SSH), TCP/80 (HTTP / WebSockify), TCP/443 (HTTPS / WordPress), TCP/3306 (MySQL)

Testing Style: Black-box external, escalating to authenticated / internal post-foothold analysis

Rules of Engagement: Testing performed against a TryHackMe-hosted training lab under TryHackMe's terms of service, within an isolated environment with no production data or third parties affected

2.2 Objectives

- Identify externally exploitable vulnerabilities in network services and the web application.
- Determine whether an attacker could gain an initial foothold on the host.
- Enumerate the compromised host for sensitive data exposure and further attack paths.
- Identify and document any indicators of compromise (IOCs) suggestive of prior, unrelated intrusion activity.
- Where IOCs are found, perform open-source threat-intelligence correlation to support attribution and incident-response reporting.
- Provide clear, actionable remediation guidance prioritized by risk.

3. Methodology

Testing followed a standard black-box penetration testing methodology consistent with industry frameworks such as PTES and the OWASP Testing Guide, extended with a lightweight digital-forensics / threat-intelligence phase once evidence of prior compromise was identified:

- Reconnaissance & Enumeration — network/service discovery, web content discovery, CMS and plugin/theme fingerprinting.
- Vulnerability Analysis — mapping discovered software versions to known CVEs.
- Exploitation — gaining an initial foothold through a validated, publicly documented exploit.
- Post-Exploitation Enumeration — reviewing configuration files, running services, and the filesystem for sensitive data and anomalies.
- Indicator Triage & Threat-Intelligence Correlation — investigating anomalies found during enumeration (suspicious services, encoded artifacts) and cross-referencing recovered indicators against open-source intelligence.

- Reporting — consolidating evidence, risk-rating each finding, and producing remediation and IOC-handling guidance.

3.1 Tools Used

- Nmap — network and service discovery
- Gobuster (dirb wordlist) — web content / directory discovery
- WPScan — WordPress core, theme, and user enumeration
- Public CVE-2024-25600 proof-of-concept exploit (third-party tool, author: K3ysTr0K3R) — Bricks Builder unauthenticated RCE
- Manual Linux enumeration (systemctl, ps, filesystem review)
- Manual decoding (hex / Base64) of recovered artifacts
- Open-source threat-intelligence lookups (blockchain explorers, sanctions and threat-actor reporting)

4. Attack Chain Overview

The diagram below summarizes the path from unauthenticated network access to full host enumeration, plus the parallel discovery track that led to identifying a pre-existing compromise. Each stage is detailed as an individual finding in Section 5.

- Stage 1 — Network and web reconnaissance identify SSH, an unusual Python/WebSockify service on port 80, HTTPS WordPress on port 443, and an exposed MySQL service on port 3306.
- Stage 2 — Directory brute-forcing and WPScan fingerprint WordPress 6.5 with the Bricks Builder theme (v1.9.5) and disclose the “administrator” username; phpMyAdmin is found exposed at a predictable path.
- Stage 3 — The publicly disclosed unauthenticated RCE in Bricks Builder (CVE-2024-25600) is exploited directly, yielding an interactive shell as the apache service account.
- Stage 4 — Reading wp-config.php from the shell discloses plaintext MySQL root credentials and WordPress secret keys/salts.
- Stage 5 — Enumeration of running services reveals a systemd unit disguised under an innocuous name (“ubuntu.service”) but reporting an unexpected status string, consistent with a masquerading persistence mechanism from a prior, unrelated compromise.
- Stage 6 — A hidden, root-owned file with a randomized filename is discovered sitting inside the web root alongside the application's own files, consistent with a dropped artifact from that same prior intrusion.
- Stage 7 — A separately recovered identifier, wrapped in multiple layers of hex and Base64 encoding, decodes to a Bitcoin wallet address found elsewhere on the host in plaintext.
- Stage 8 — Open-source threat-intelligence correlation on that wallet address links it to a Bitcoin address publicly attributed to a sanctioned individual associated with the LockBit ransomware group, indicating the earlier compromise may be tied to known ransomware-affiliated threat activity.

5. Detailed Findings

5.1 Network & Service Reconnaissance

Severity: INFO

An Nmap service/version scan identified four notable open TCP ports: SSH, an HTTP service on port 80 identified as a Python http.server / WebSockify instance (unusual for a standard WordPress deployment and

often associated with VNC/remote-console proxying), an HTTPS Apache service on port 443 serving the actual WordPress site, and a directly reachable MySQL service on port 3306.

```
$ nmap -sC -sV 10.48.170.53
PORT     STATE SERVICE  VERSION
22/tcp   open  ssh      OpenSSH 8.2p1 Ubuntu 4ubuntu0.11
80/tcp   open  http     Python http.server 3.5 - 3.10
|_http-server-header: WebSockify Python/3.8.10
443/tcp  open  ssl/http Apache httpd
|_http-generator: WordPress 6.5
|_http-title: Brick by Brick
3306/tcp open  mysql    MySQL (unauthorized)
```

Impact: The combination of an exposed database port and an atypical WebSockify listener widens the attack surface beyond the web application itself and warrants dedicated review; WebSockify services in particular are commonly paired with remote console/VNC access and should not be reachable from the open internet.

Recommendation: Restrict MySQL (3306) to localhost or a private management network via firewall rules; identify the purpose of the WebSockify listener on port 80 and remove or firewall it if not intentionally required for the application.

5.2 Web Content Discovery & Exposed Administrative Interfaces

Severity: MEDIUM

Directory brute-forcing against the target revealed the standard WordPress admin paths as well as a live phpMyAdmin instance reachable at a predictable, default path — a common and heavily targeted entry point for credential attacks against the database layer.

```
$ gobuster dir -u https://bricks.thm/ -w /usr/share/wordlists/dirb/common.txt -k
/admin                (Status: 302) --> https://bricks.thm/wp-admin/
/dashboard            (Status: 302) --> https://bricks.thm/wp-admin/
/phpmyadmin           (Status: 301) --> https://bricks.thm/phpmyadmin/
/wp-admin             (Status: 301)
/wp-content           (Status: 301)
/xmlrpc.php           (Status: 405)
```

Recommendation: Remove or restrict access to phpMyAdmin from the public internet (VPN/IP allow-listing or a management-only network segment), disable XML-RPC if unused, and avoid deploying database administration tools on internet-facing hosts.

5.3 WordPress User Enumeration

Severity: MEDIUM

WPScan's passive and active enumeration techniques (RSS generator metadata, the WP-JSON REST users endpoint, author ID brute-forcing, and login error-message differences) disclosed a valid WordPress username, “administrator,” without any authentication.

```
[i] User(s) Identified:
[+] administrator  (Rss Generator / Wp Json Api / Author Id Brute Forcing / Login
Error Messages)
```

Recommendation: Disable the WP-JSON users endpoint for unauthenticated requests, use generic login error messages, and enforce account lockout / rate limiting on wp-login.php.

5.4 Critical: Unauthenticated Remote Code Execution in Bricks Builder Theme (CVE-2024-25600)

Severity: CRITICAL

The site was running WordPress 6.5 with the Bricks Builder theme, version 1.9.5. This version is affected by CVE-2024-25600, a publicly disclosed unauthenticated Remote Code Execution vulnerability in the theme's element-rendering REST endpoints. The flaw allows an unauthenticated attacker to obtain a valid request nonce from the public site and then supply crafted PHP execution logic through the theme's query/element settings, resulting in arbitrary command execution in the context of the web server.

A publicly available proof-of-concept tool for this CVE was used to confirm and exploit the vulnerability, which established an interactive pseudo-shell against the target.

```
$ python3 CVE-2024-25600.py -u https://bricks.thm/
[*] Checking if the target is vulnerable
[+] The target is vulnerable
[*] Initiating exploit against: https://bricks.thm/
[+] Interactive shell opened successfully
Shell> id
uid=1001(apache) gid=1001(apache) groups=1001(apache)
```

Impact: Complete compromise of the web application and the ability to execute arbitrary commands as the web server user with zero authentication required — the most severe class of web application vulnerability. This finding was the entry point for the entire remainder of the assessment.

Recommendation: Update the Bricks Builder theme immediately to a patched release, or remove it if not actively required. Ensure all themes and plugins are subject to the same patch-management cadence as WordPress core. Consider a Web Application Firewall rule to detect and block exploitation attempts against the affected REST routes (/wp-json/bricks/v1/render_element and the ?rest_route= equivalent) as a compensating control while patching is completed.

5.5 Sensitive Configuration Disclosure — wp-config.php

Severity: CRITICAL

With shell access as the apache user, the WordPress configuration file was readable directly from disk. This file disclosed the MySQL database username and password in plaintext, along with all eight WordPress authentication unique keys and salts — values that, if leaked, allow forgery of valid WordPress authentication cookies for any user without knowing their password.

```
Shell> cat wp-config.php
define( 'DB_NAME', 'wordpress' );
define( 'DB_USER', 'root' );
define( 'DB_PASSWORD', 'lamp.sh' );
define( 'DB_HOST', 'localhost' );
define( 'AUTH_KEY', '...' ); define( 'SECURE_AUTH_KEY', '...' );
define( 'LOGGED_IN_KEY', '...' ); define( 'NONCE_KEY', '...' ); [+ 4 more salts]
```

Impact: The disclosed credentials use the MySQL root account for the application (a severe violation of least privilege on their own), and the password value — a short, dictionary-style string — offers minimal resistance to brute-force or credential-stuffing attacks. Combined with Finding 5.6 (an exposed MySQL port), this gives a remote, authenticated-to-the-database attacker full read/write control of all WordPress data, and combined with the leaked secret keys, the ability to forge authenticated administrator sessions directly.

Recommendation: Rotate the database password and all WordPress secret keys/salts immediately. Create a dedicated, least-privilege MySQL account for the WordPress application rather than using the root account. Ensure wp-config.php is not web-readable and is excluded from any backup or deployment artifact that could be exposed.

5.6 Exposed MySQL Service Combined with Weak/Reused Database Credentials

Severity: HIGH

Nmap confirmed MySQL (port 3306) was reachable directly from the network rather than being restricted to localhost. Combined with the plaintext root credentials recovered in Finding 5.5, this represents a direct, low-effort path to full database compromise that does not require re-exploiting the web application at all.

Impact: An attacker who obtains the credentials in 5.5 by any means (this compromise, a backup leak, a future breach) can connect to the database directly and read, modify, or delete all application data without further exploitation.

Recommendation: Bind MySQL to localhost only (bind-address = 127.0.0.1) unless remote database access is a genuine business requirement, in which case restrict it to specific management IPs via firewall and require TLS.

5.7 Evidence of Pre-Existing Compromise — Masquerading Persistence Service

Severity: CRITICAL

While enumerating running services from the foothold shell, a systemd unit named ubuntu.service was observed active and running — a name deliberately chosen to blend in with legitimate operating-system services. However, its reported status string was not consistent with any standard Ubuntu system service. This strongly suggests the unit is a disguised persistence mechanism installed by a separate actor prior to this assessment, rather than a legitimate part of the base operating system.

```
Shell> systemctl list-units --type=service --state=running
  ubuntu.service   loaded active running TRYHACK3M
```

Impact: A systemd service masquerading as a benign OS component is a classic persistence and evasion technique used by real-world intrusion actors (including ransomware affiliates) to maintain access and blend into normal system activity, evading casual administrator review. Its presence indicates the host was likely compromised by a threat actor independent of this assessment, and that persistence had already been established.

Recommendation: Treat this as an active incident: isolate the host from the network, capture forensic evidence of the unit file and its origin (creation timestamp, associated binary, parent process), inventory all systemd units for further disguised entries, and rebuild the host from a known-good image rather than attempting in-place remediation.

5.8 Planted Hidden Artifact File in Web Root

Severity: HIGH

A directory listing of a web-application folder revealed a file with a randomized, hash-like filename, owned by root, sitting alongside the application's own files (which are owned by the apache/web-server account). A file with these characteristics — root ownership inside a directory otherwise owned by the web server user, a filename resembling a hash rather than any legitimate application asset — is consistent with an artifact deliberately dropped by an intruder, such as a webshell remnant, a marker file, or staged exfiltration data.

```
Shell> ls -la
drwxr-xr-x  7 apache apache  4096 Sep  3 09:00 .
-rw-r--r--  1 apache apache   523 Apr  2  2024 .htaccess
-rw-r--r--  1 root   root      43 Apr  5  2024
650c844110baced87e1606453b93f22a.txt
```

Impact: The ownership mismatch (root, rather than the web server account that owns everything else in the directory) indicates this file was placed by someone with a higher privilege level than the web application itself possesses — further corroborating that a separate, more privileged compromise of the host had already occurred.

Recommendation: Preserve this file and its metadata (timestamps, permissions) as forensic evidence, do not modify or delete it before analysis, and include it in the incident-response evidence package alongside the persistence-service finding in 5.7.

5.9 Obfuscated Cryptocurrency Wallet Artifact Recovered from Host

Severity: HIGH

A separate identifier recovered during enumeration was found to be a hexadecimal-encoded string. Decoding it revealed a Base64 string, which itself decoded to a second layer of Base64 — a hex → Base64 → Base64 obfuscation chain uncommon in legitimate application data but frequently used by malware and intrusion tooling to hide configuration values (such as command-and-control addresses or payment/wallet destinations) from casual inspection or simple string-matching detection. Fully decoded, the value resolved to a Bitcoin wallet address that also appeared in plaintext elsewhere on the host.

```
# Layer 1 (as found): hex-encoded string
5757314e65474e5962484a4f656d787457544e424e574648555446684d3070735930684b616c70...

# Layer 2: hex-decode -> Base64 string -> Base64-decode
# Layer 3: Base64-decode again -> plaintext

Decoded result:
bc1qyk79fcp9hd5kreprce89tkh4wrtl8avt4l67qa
```

Impact: Multi-layer encoding of a payment address is a deliberate evasion technique, not something a legitimate WordPress site or plugin would do. This is a strong indicator that the host was used, or was intended to be used, in connection with a financially motivated compromise (e.g., ransomware, extortion, or cryptojacking), reinforcing the persistence and planted-artifact findings above.

Recommendation: Preserve the encoded and decoded values as part of the forensic record. Do not interact with the wallet address (no test transactions) as this can tip off an active threat actor and may have legal implications given the sanctions status identified in Finding 5.10. Report the indicator to the appropriate internal security/legal stakeholders.

5.10 Threat-Intelligence Correlation — Wallet Linked to Sanctioned LockBit-Affiliated Actor

Severity: CRITICAL

The Bitcoin address recovered in Finding 5.9 was cross-referenced against open-source threat-intelligence and blockchain-attribution reporting. Public reporting associates this address with Ivan Gennadievich Kondratiev (alias “Bassterlord”), a Russian national who has been publicly identified in connection with the LockBit ransomware operation and whose associated wallet addresses have been the subject of sanctions designations.

```
Indicator: bc1q5jqgm7nvrhaw2rh2vk0dk8e4gg5g373g0vz07r
Attribution (OSINT): Linked to Ivan Gennadievich Kondratiev ("Bassterlord")
Associated Threat Group: LockBit ransomware operation
Status: Publicly reported as a sanctioned/designated address
```

Note: This is a threat-intelligence / OSINT correlation finding rather than a technical vulnerability, and is included because it materially changes the incident-response priority of Findings 5.7–5.9 — from “suspicious artifacts” to “likely evidence of ransomware-affiliated threat actor activity.”

Impact: If confirmed in a real environment, this would elevate the incident from a standard web-application compromise to a suspected ransomware-affiliate intrusion, with corresponding obligations around incident-response scope, law-enforcement notification, and regulatory/sanctions-compliance reporting depending on jurisdiction.

Recommendation: Escalate immediately to incident-response leadership and legal/compliance stakeholders. Engage law enforcement or a national CERT where appropriate given the sanctions nexus. Do not attempt to independently contact, transact with, or further investigate the wallet or associated infrastructure beyond passive OSINT review, and preserve a full chain-of-custody record of how the indicator was discovered.

5.11 VPN / Tunneling Utilities Present Under System Network Manager Path

Severity: MEDIUM

Enumeration of /lib/NetworkManager identified a full set of OpenVPN client/service binaries and configuration directories present on the host, including nm-openvpn-service and an associated system-connections directory. While NetworkManager's OpenVPN plugin is not unusual on a standard Linux desktop image, its presence and configuration state on an internet-facing server warrants review, particularly given the other indicators of compromise on this host — VPN/tunneling capability is a common mechanism for maintaining covert outbound connectivity or exfiltration channels.

```
Shell> ls /lib/NetworkManager
VPN  conf.d  dispatcher.d  nm-openvpn-auth-dialog  nm-openvpn-service
nm-openvpn-service-openvpn-helper  nm-pptp-auth-dialog  nm-pptp-service
system-connections
```

Recommendation: Review the contents of system-connections for any configured VPN profiles pointing to unfamiliar endpoints, and confirm whether OpenVPN/PPTP client capability is a legitimate requirement for this server; if not, remove the packages to reduce the host's available tunneling/evasion capability.

6. Risk Summary & Business Impact

This engagement combined two distinct but related risk narratives. First, a straightforward but severe technical vulnerability — an unauthenticated RCE in a widely used WordPress page-builder theme — gave immediate, unauthenticated command execution on the host. Second, and more unusually, post-exploitation enumeration surfaced credible evidence that the host had already been compromised by a separate, financially motivated threat actor prior to this engagement, with indicators pointing toward LockBit-affiliated ransomware activity.

In a real production environment, this combination would represent a severe incident: an actively exploitable entry point that could allow re-compromise even after cleanup, layered on top of an existing, likely undetected intrusion with ransomware-actor ties. The business impact of leaving either condition unaddressed includes data theft, ransomware deployment, regulatory and sanctions-compliance exposure, and reputational damage.

7. Recommendations Summary

7.1 Immediate (Critical / High Priority)

- Patch or remove the Bricks Builder theme to a version not affected by CVE-2024-25600; deploy a compensating WAF rule in the interim.
- Treat Findings 5.7–5.10 as an active incident: isolate the host, preserve forensic evidence, and engage incident-response/legal stakeholders given the sanctions nexus.
- Rotate the MySQL root password and all WordPress secret keys/salts disclosed in wp-config.php; move to a dedicated least-privilege database account.
- Restrict MySQL (3306) to localhost or a private management network.

7.2 Short-Term (Medium Priority)

- Remove or restrict public access to phpMyAdmin and other administrative interfaces.
- Disable WordPress user enumeration via the REST API and generic-ize login error messages.
- Review and remove unnecessary VPN/tunneling client capability from the host unless explicitly required.
- Identify and justify (or remove) the WebSockify listener on port 80.

7.3 Long-Term (Process & Hygiene)

- Establish a recurring patch-management cadence for WordPress core, themes, and plugins, with particular attention to page-builder and form-handling plugins/themes, which are frequent RCE targets.
- Deploy file-integrity monitoring on web roots and system service directories to catch planted artifacts and masquerading persistence mechanisms earlier.
- Implement centralized logging/alerting for new systemd unit creation and unexpected outbound network connections.
- Conduct periodic penetration testing and, where warranted, a dedicated compromise assessment to catch dormant or pre-existing intrusions.

8. Conclusion

This engagement demonstrated a critical, publicly known unauthenticated RCE vulnerability in the Bricks Builder WordPress theme, successfully exploited to gain command execution on the target host. Post-exploitation enumeration went further than a typical vulnerability assessment, surfacing credible evidence — a masquerading persistence service, a root-owned planted artifact, and a multi-layer-encoded cryptocurrency wallet address linked via open-source threat intelligence to a sanctioned individual associated with the LockBit ransomware group — that this host had already been compromised by a separate threat actor before this assessment began.

Every technical finding in this report is remediable with established best practices; the threat-intelligence findings additionally require an incident-response and compliance-driven response rather than a purely technical one. This report and the accompanying evidence are provided to support both remediation planning and incident-response escalation for the intended recipient.

Note: This engagement was conducted against the “TryHack3M: Bricks Heist” room on TryHackMe (thm.io), a legal, purpose-built training platform for practicing offensive security and threat-intelligence skills. The “prior

compromise” and threat-actor attribution narrative is part of the room's designed scenario. This report is presented as a portfolio / methodology demonstration and reflects the tester's own analysis, findings, and remediation guidance rather than an assessment of a live production system or a real ransomware incident.

Appendix A — Evidence Index

- port_scan.txt — network/service scan output
- gobuster.txt — web content discovery output
- userfound.txt — WPScan core, theme, and user enumeration output
- CVE-2024-25600.py / shell.txt — public PoC exploit and captured exploitation session (initial foothold as apache)
- DBuserPass.txt — disclosed wp-config.php contents (database credentials and WordPress secret keys)
- suspiciousProcess.txt / servicenameAffilitadToSuspiciousProcess.txt — masquerading systemd persistence service
- hidden.txt — planted, root-owned hidden file discovered in the web root
- walletaddr.txt — recovered cryptocurrency wallet address (plaintext and encoded forms)
- ThreatGroup.txt — open-source threat-intelligence attribution of the recovered wallet address
- logfile.txt — VPN/tunneling utilities identified under /lib/NetworkManager